Home Cybersecurity Lab — Full Build Documentation

Project Overview

Built a professional-grade cybersecurity home lab from scratch using enterprise networking concepts including VLAN segmentation, firewall routing, and SIEM monitoring. The lab provides a safe, isolated environment for practicing offensive and defensive security techniques.

Hardware Used

Device	Model	Role
PC1 (Server)	Lenovo ThinkCentre M910q	Proxmox VE hypervisor host
PC2	Personal workstation	Management and monitoring
Switch	TP-Link TL-SG108E	Managed switch with VLAN support
Router	Rogers Ignite Gateway	ISP internet connection
Access Point	TP-Link Omada EAP723	Segmented WiFi networks
USB NIC	UGREEN USB 3.0 Gigabit	Dedicated WAN interface

Software Stack

Software	Version	Purpose
Proxmox VE	9.1.1	Type-1 hypervisor
pfSense	2.8.1	Firewall and router
Kali Linux	2025.4	Attack machine
Ubuntu Server	Latest	Linux target
Windows 11	Latest	Windows target
Wazuh	4.14.2	SIEM and security monitoring

Network Architecture

```
Internet (Rogers ISP)
         |
Rogers Gateway (Bridge Mode)
         |
```

```
UGREEN USB NIC -> Proxmox PC1
         |
pfSense WAN (x.x.x.x)
         |
pfSense LAN (192.168.x.1)
         |
Isolated Lab Network (192.168.x.0/24)
         |
    +---------------+-----------------+---------------+
  Kali Linux    Ubuntu Server    Windows 11     Wazuh SIEM
  192.168.x.10  192.168.x.20     192.168.x.30   192.168.x.50
```

VLAN Design

VLAN	Name	Purpose	Subnet
1	Management	Proxmox management, PC2 access	x.x.x.0/24
20	Lab	Isolated lab network for VMs	192.168.x.0/24
40	WiFi/Guest	Guest WiFi network	192.168.x.0/24
50	IoT	IoT device network	192.168.x.0/24

Phase 1 — Proxmox VE Setup

What I Did

Installed Proxmox VE on the ThinkCentre M910q and configured the network interfaces to support VLAN trunking. The physical NIC (nic0) was configured as a trunk port carrying all VLANs tagged.

Key Concepts Learned

- Linux bridges act like virtual switches inside Proxmox
- VLAN sub-interfaces (nic0.1, nic0.20, etc.) filter traffic by VLAN tag
- Each bridge connects VMs to a specific network segment

Network Interfaces Created

Interface	Type	Purpose
nic0	Physical NIC	Trunk port to switch
nic0.1	VLAN sub-interface	VLAN 1 management traffic
nic0.20	VLAN sub-interface	VLAN 20 lab traffic
nic0.40	VLAN sub-interface	VLAN 40 WiFi traffic
nic0.50	VLAN sub-interface	VLAN 50 IoT traffic

Interface	Type	Purpose
vmbr0	Linux bridge	Management network (x.x.x.x)
vmbr20	Linux bridge	Isolated lab network
vmbr30	Linux bridge	WAN (connected to USB NIC)
vmbr40	Linux bridge	Guest WiFi network
vmbr50	Linux bridge	IoT network

Phase 2 — Managed Switch Configuration

What I Did

Configured the TL-SG108E managed switch with 802.1Q VLANs to segment network traffic. Port 2 (connected to Proxmox) was configured as a trunk port carrying all VLANs tagged.

Key Concepts Learned

- Tagged ports carry multiple VLANs — the receiving device must understand VLAN tags
- Untagged ports strip the VLAN tag — the connected device doesn't know about VLANs
- PVID (Port VLAN ID) determines which VLAN untagged traffic belongs to
- Every port must belong to at least one VLAN on the TL-SG108E

Switch Port Configuration

Port	Connected Device	VLAN 1	VLAN 20	VLAN 40	VLAN 50
Port 1	(unused/ISP)	Untagged	—	—	—
Port 2	Proxmox PC1	Tagged	Tagged	Tagged	Tagged
Port 3	TP-Link AP	Untagged	—	Tagged	Tagged
Port 4	PC2	Untagged	—	—	—

Challenges Encountered

- The TL-SG108E drops connections when modifying VLAN 1 since management traffic is on VLAN 1
- Solution: Configure all other VLANs first, then modify VLAN 1 last
- Switch config must be saved before modifying VLAN 1 to survive disconnection
- Hardware V6 with 2023 firmware resolved older Save Config bugs

Phase 3 — WAN Interface Setup

What I Did

Added a UGREEN USB 3.0 Gigabit Ethernet adapter (ASIX AX88179 chipset) to Proxmox as a dedicated WAN interface. Connected directly to the Rogers gateway LAN port, bypassing the switch entirely for WAN traffic.

Why This Approach

- Rogers Ignite gateway is configured in bridge mode, passing the public IP directly to pfSense
- Using a dedicated WAN NIC eliminates VLAN complexity for internet access
- pfSense gets a direct connection to Rogers without double NAT complications
- Cleaner separation between WAN and LAN traffic

Configuration

- USB NIC detected as enx_____ in Proxmox
- Created vmbr30 bridge attached to USB NIC
- pfSense WAN interface assigned to vmbr30
- pfSense WAN gets DHCP IP (x.x.x.x) from Rogers gateway

Phase 4 — pfSense Firewall Configuration

What I Did

Deployed pfSense 2.8.1 as the primary firewall and router for the lab network. pfSense controls all traffic between the isolated lab network and the internet.

Interface Assignments

pfSense Interface	Proxmox Bridge	IP Address	Purpose
WAN (vtnet0)	vmbr30	x.x.x.x (DHCP)	Internet uplink
LAN (vtnet1)	vmbr20	192.168.x.1/24	Lab network gateway
GUEST (vtnet3)	vmbr40	192.168.x.1/24	Guest WiFi gateway
IOT (vtnet4)	vmbr50	192.168.x.1/24	IoT network gateway

DHCP Servers Configured

Interface	Range	Purpose
LAN	192.168.x.100-200	Lab VM addresses
GUEST	192.168.x.10-200	Guest WiFi devices
IOT	192.168.x.10-200	IoT devices

Firewall Rules

- LAN → Any: Allow all lab VM traffic outbound
- GUEST → Any: Allow guest internet access
- IOT → Any: Allow IoT internet access
- Blocked: Lab VMs cannot reach home management network

Key Concepts Learned

- pfSense uses NAT to share one public IP across all lab devices
- Firewall rules are evaluated top to bottom — first match wins
- Source must be set to interface net not interface address for proper traffic filtering
- Each interface needs its own DHCP server and firewall rules

Phase 5 — Static IP Assignments

What I Did

Assigned static IPs to all lab VMs to ensure consistent addressing for Wazuh agent communication and lab exercises.

Method Used

Used pfSense DHCP static mappings (MAC address reservations) instead of configuring static IPs inside each VM — more reliable and centrally managed.

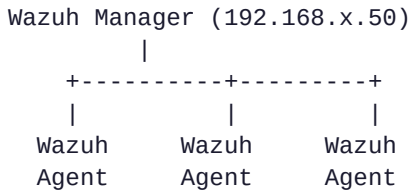
VM	MAC Address	Static IP
Kali Linux	xx:xx:xx:xx:xx:xx	192.168.x.10
Ubuntu Server	xx:xx:xx:xx:xx:xx	192.168.x.20
Windows 11	xx:xx:xx:xx:xx:xx	192.168.x.30
Wazuh SIEM	xx:xx:xx:xx:xx:xx	192.168.x.50

Phase 6 — Wazuh SIEM Deployment

What I Did

Deployed Wazuh 4.14.2 as the SIEM platform to monitor all lab VMs. Installed Wazuh agents on Kali, Ubuntu, and Windows 11 to ship logs and security events to the central Wazuh server.

Architecture



```
(Kali)   (Ubuntu)  (Win11)
x.10      x.20      x.30
```

Agent Installation

Linux (Kali/Ubuntu):

```
wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/\
    wazuh-agent_4.14.2-1_amd64.deb
sudo WAZUH_MANAGER='192.168.x.50' WAZUH_AGENT_NAME='kali' \
    dpkg -i ./wazuh-agent_4.14.2-1_amd64.deb
sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent
```

Windows 11:

```
Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/\
    wazuh-agent-4.14.2-1.msi -OutFile $env:tmp\wazuh-agent.msi
msiexec.exe /i $env:tmp\wazuh-agent.msi /q \
    WAZUH_MANAGER="192.168.x.50" WAZUH_AGENT_NAME="win11"
NET START WazuhSvc
```

Challenges Encountered

- SELinux on Wazuh VM (Amazon Linux) blocking port 1514
- Solution: Set SELinux to permissive mode and open port 1514
- Cloud-init on Ubuntu overriding network configuration
- Solution: Deleted cloud-init netplan file and disabled cloud-init network management

Phase 7 — WiFi Network Segmentation

What I Did

Configured the TP-Link Omada EAP723 access point with three separate SSIDs, each mapped to a different VLAN for network segmentation.

SSID Configuration

SSID	VLAN	Subnet	Purpose
Home	1	x.x.x.x	Primary home network
Guest	40	192.168.x.x	Visitor access — isolated
IoT	50	192.168.x.x	Smart home devices — isolated

Key Concepts Learned

- AP trunk port must be tagged for each VLAN it serves
- Each SSID maps to a VLAN ID which gets tagged by the AP
- pfSense handles DHCP and routing for each VLAN segment

- Guest and IoT networks are completely isolated from the lab and home networks

Challenges and Solutions

Challenge	Root Cause	Solution
Lost switch access	Modifying VLAN 1 drops management connection	Configure all other VLANs first, modify VLAN 1 last
Rogers bridge mode initially unstable	Rogers Ignite gateway required multiple configuration attempts for stable bridge mode	Persisted with bridge mode configuration until stable; added dedicated USB NIC for clean WAN path
pfSense LAN unreachable	tap interface not attached to bridge after reboot	Added post-up script to interfaces file
Wazuh agents not sending data	SELinux blocking port 1514	Set SELinux to permissive, opened port 1514
Ubuntu cloud-init overriding IP	cloud-init netplan file taking priority	Deleted 50-cloud-init.yaml and disabled cloud-init networking
pfSense HOME interface conflict	x.x.x.x conflict on shared vmbr0 bridge	Removed HOME interface, used separate approach

Skills Demonstrated

- Network segmentation using VLANs on managed switches
- Virtualization with Proxmox VE type-1 hypervisor
- Firewall configuration with pfSense including NAT, DHCP, and firewall rules
- Linux networking including bridges, VLAN sub-interfaces, and routing
- SIEM deployment with Wazuh including agent installation and configuration
- Troubleshooting complex network issues across multiple layers (L2/L3)
- Security monitoring with centralized log collection

Future Improvements

- Complete Wazuh agent communication fix
- Add pfSense IDS/IPS with Suricata
- Configure Wazuh active response rules
- Add vulnerability scanning with OpenVAS
- Practice CTF exercises between Kali and target VMs
- Add Windows Active Directory domain
- Configure VPN access to lab from outside

Lessons Learned

This project taught me that real-world networking is far more complex than theory. Every component interacts with every other component — a change to the switch affects the firewall, which affects the VMs, which affects monitoring. The most valuable skill developed was systematic troubleshooting: isolating each layer and testing methodically rather than making random changes.

The lab now provides a realistic environment that mirrors enterprise security infrastructure at a fraction of the cost.